Day 47 – SOAP Adapter, WSDL & WS-Security

Class date: 4th September 2026 | Module: Adapters (SOAP), API-based Integrations

Two small corrections to your handwritten notes:
1) You wrote SoapUI is a **"tree open source tool."** This looks like a slip of the pen for **"free, open-source tool."** SoapUI's core edition genuinely is free and open source (there is also a separate paid ReadyAPI/SoapUI Pro edition with extra features, but the base tool used for this kind of SOAP/REST testing is free).
2) Your WSDL interlinking diagram listed the chain roughly as "port → operations → bindings → message → element," and separately "Service → port → binding → operations → message." The precise top-down containment in a real WSDL file is: **Service** → **Port** → **Binding** → **PortType (Operations)** → **Message** → **Element (defined in the Schema/Types section).** Your second line was actually the correct order – Section 3 below lays this out clearly with the reasoning for why it flows in that direction.

1. What is the SOAP Adapter for? (start here first)

If you want to call any **SOAP-based web service** or **SOAP-based API**, you use the **SOAP Adapter**. More generally: whenever you need any **web-service-based communication**, the SOAP Adapter is the one to use.

API-based integrations (which power most real-time integrations today) mainly fall into three styles: **SOAP, REST, and OData.**

Historical context: before REST-style APIs became common, systems that needed to expose functionality over a network used to expose it as a **Web Service** instead – this is what SOAP was originally built for, and is why SOAP is still very common with older/legacy systems today.

Prerequisite for the SOAP Adapter: a **WSDL file** (Web Service Definition Language). Without the WSDL, you cannot configure a SOAP receiver adapter against a target service.

2. What is a WSDL file?

A WSDL is nothing but the **schema/contract of a service** – it describes exactly **what operations** can be called, and **what the endpoint** is.

Example used in class – a Calculator web service: its WSDL exposes operations like **Add, Multiply, Subtract,** etc. The demo WSDL was taken from a public sample site (**dneonline.com**) – a free calculator SOAP web service commonly used for SOAP training/demos – viewed and downloaded to the local system, then saved with a **.wsdl** extension.

Why the WSDL file matters: every WSDL file starts with the root element **<wsdl:definitions>**, which contains (among other things) a **<wsdl:types>** section wrapping an embedded schema, e.g.:

```
... element definitions ... ...
```

That embedded **<s:schema>...</s:schema>** block is an **XSD (XML Schema Definition)** – XSD is nothing but a schema that defines the structure/data types used by the service.

3. WSDL structure – how the pieces link together (corrected)

A WSDL's **<wsdl:service>** element can contain one or more **<wsdl:port>** elements, each with its own **name** and **binding** attribute, plus a **soap:address location** (the actual endpoint URL for that port):

- **Port** – tells you which specific endpoint you are connecting to (the "location" is the actual address/endpoint you're hitting).
- **Binding** (**SOAP Binding**) – used to bind/attach the actual set of operations to that port, specifying the protocol and message format details.

The correct top-down chain, as confirmed against SAP/WSDL documentation:

```
Service -> Port -> Binding -> (PortType) Operations -> Message -> Element (defined inside the Types/Schema section)
```

All of these are **interlinked**: the Service exposes a Port; the Port references a Binding; the Binding attaches concrete protocol details to a set of Operations; each Operation refers to Input/Output Messages; each Message is built from Elements (the field names/structure), which are ultimately defined in the **Types** section using XSD. In plain XML terms: **every individual field is called an Element.**

To call any target system's SOAP service, you need from them: the WSDL file itself, which in turn gives you the **Service, Port, Binding, and Operations**.

4. SOAP vs REST – structure & sync/async

- Calling a **Web Service (API)** in SOAP means calling against a **fixed, defined structure** (the WSDL contract).
- **REST** does not enforce any single fixed structure/contract in the same way.
- **SOAP** uses a strictly defined message format and **only accepts XML content.**

Synchronous vs. Asynchronous (from the WSDL's Message definitions): if a Message has both **Input and Output** defined, the operation is **Synchronous** (Request-Reply/two-way). If a Message has **only Input** defined (no Output), the operation is **Asynchronous** (one-way, fire-and-forget). Input = the Request; Output = the Response.

URL and structure are both defined inside the WSDL – this is what makes SOAP calls predictable and strongly typed compared to REST.

5. SoapUI – third-party testing tool

SoapUI is a free, open-source third-party tool (corrected above) commonly used for testing **SOAP-based APIs**, and is also widely used for **regression testing** of web services.

Demo steps followed in class:

1. **Step 1:** Create a new project in SoapUI.
2. **Step 2:** Upload/import the WSDL file (the Calculator WSDL).

Opening the imported WSDL in SoapUI exposed two ports: **CalculatorSoap** and **CalculatorSoap12** – both expose the **same operations** (Add, Divide, Multiply, Subtract), just under two different SOAP binding versions/ports. Each operation shows a **fixed, ready-made request structure/template**, generated directly from the WSDL.

Mandatory tags whenever sending SOAP data: every SOAP request/response must include these, otherwise the interface call will fail:

1. **SOAP Envelope** – the mandatory outer wrapper tag for the entire message.
2. **SOAP Header** – optional metadata about the message (e.g. security/auth info).

3 **SOAP Body** – contains the actual **XML payload** (the real request/response data).

SOAP is mainly used for **legacy systems**. General usage flow in SoapUI: **Import (WSDL file)** → **select Operation** → **fill Payload** → **hit URL.**

6. Calling a SOAP web service from SAP CPI – Receiver SOAP Adapter

Comparing the Connection tab fields for a plain REST/API call vs. a SOAP call:

- **Generic API:** Endpoint URL → Authentication.
- **SOAP:** Endpoint URL → Authentication → **WSDL** (the extra piece SOAP needs on top of a plain API call).

Receiver SOAP Adapter – Connection tab fields, in order:

#	Field	Notes
1	Address	The endpoint URL.
2	Proxy Type	Internet or On-Premise (same concept as HTTP/SFTP receiver adapters).
3	URL to WSDL	Upload the WSDL file itself (e.g. Calculator.wsdl).
4	Service	Parsed automatically from the uploaded WSDL.
5	Endpoint	The Binding, again parsed from the WSDL.
6	Operation Name	Which specific operation (e.g. Add) is being called.
7	Authentication	None (common for legacy systems), Certificate, or SAML/Bearer.
8	Timeout	e.g. 6,000 ms in the demo.
9	Keep-Alive	Enabled in the demo – keeps the underlying connection open for reuse.
10	Compress Message	Enabled in the demo – compresses the payload for transfer.
11	Allow Chunking	Not enabled in the demo.
12	Return HTTP Response Code as Header	Not enabled in the demo.

Good news for developers: you do **not** need to manually add the SOAP Envelope/Header/Body mandatory tags yourself in the message mapping – the **SOAP Adapter itself adds and removes them automatically** at runtime.

7. WS-Security

WS-Security matters specifically when working with a SOAP web service in **legacy applications** that expect **security headers** inside the SOAP message itself, rather than (or in addition to) transport-level authentication.

If a requirement says the WSDL/target system needs security headers, WS-Security must be configured (set to **Secure**, not left at the default "None"). The WS-Security tab of the SOAP Receiver Adapter offers

two configuration approaches:

- **Based on Policies in WSDL** – the security settings (per the WS-Policy standard) are already defined inside the target's own WSDL, and CPI just follows them.
- **None** – no WS-Security applied.
- **Via Manual Configuration in Channel** – you configure it yourself in CPI, with sub-options:
- **Password type**: Hashed Password, None, or Plain Text Password.
- **Signing and Encryption**: None, Sign and Encrypt Message, or Sign Message (dynamic, one-way options only).

8. SOAP Sender Adapter

The SOAP Sender Adapter's Connection tab is structured much like the HTTP Sender Adapter, with a few SOAP-specific additions:

- **Address** – the relative endpoint on which CPI will expose/receive the incoming SOAP requests.
- **Service Definition – Use WS-Addressing** – accepts addressing information from message headers provided by the sender at runtime.
- **Message Exchange Pattern**: **Request-Reply** (two-way, **Synchronous**) or **One-Way** (**Asynchronous**-based connection).

Processing Settings – Robust vs. WS Standard

Setting	Behaviour
Robust	Executes the full iFlow synchronously before responding. On success, sender gets HTTP 2xx. If any error occurs, the error content is sent back to the sender system as an HTTP 500 – the sender can see and react to the failure.
WS Standard	Sender is acknowledged as soon as the message is fully received by CPI (before full processing finishes). If an error occurs afterward during processing, that error will NOT be sent back to the sender system – the sender has no way to know the message ultimately failed unless a separate persistence/queueing mechanism is used.

Endpoint URL options: can be exposed either as a plain WSDL, or a **WSDL without policies** (i.e. the WSDL is generated/available without any WS-Policy security requirements baked in).

Quick-recall summary

- SOAP Adapter is used for any SOAP-based web service/API call; requires a **WSDL** as prerequisite.
- WSDL chain: **Service** → **Port** → **Binding** → **Operations (PortType)** → **Message** → **Element** (from the Types/XSD section).
- Message with Input+Output = **Synchronous**; Input only = **Asynchronous**.
- SoapUI = free, open-source tool for testing/regression-testing SOAP APIs.

- SOAP mandatory tags (Envelope, Header, Body) are auto-added/removed by the SOAP Adapter – no manual mapping needed.
- WS-Security: Based on Policies in WSDL / None / Manual Configuration (Password type + Signing/Encryption options).
- SOAP Sender Processing: **Robust** = errors reported back to sender (HTTP 500); **WS Standard** = errors NOT reported back once the message is accepted.